SQL review records

Four historical requests / 3 November 2026

These records preserve actual outcomes from synthetic SQL submitted during development. They are reference documents, not preloaded application content. Times below are UTC. The first attempt failed; it is retained alongside successful and deterministic checks.

Record index

Record	Time (UTC)	Result	Semantic review
1	05:49:29	incomplete	failed
2	05:50:59	needs_review	completed
3	05:53:25	passed	completed
4	05:56:16	needs_review	skipped

The model was deepseek-v4.1-flash-1017. Requests 1 and 2 did not retain public generation parameters in their report metadata; no settings are reconstructed for those records. Requests 3 and 4 record thinking disabled, max_tokens 4096 and timeout 90 seconds. Monetary cost and independent human fix correctness were not measured.

Record 1: incomplete

ID: 73ebc604-3c49-43d5-80b6-9331b933f527
Created: 2026-11-03T05:49:29.434427Z
Model: deepseek-v4.1-flash-1017 / Prompt: policy-v2
Elapsed: 90.452 seconds / Semantic review: failed

Model connection failed or timed out. Usage could not be confirmed.

Submitted SQL

```
SELECT SUM(total_amount) AS recognized_revenue
FROM orders;
```

No accepted findings were stored. For an incomplete report this does not mean the query passed.

Token usage: not reported

Record 2: needs_review

ID: be9d1c7c-aed9-4ea0-8118-645b905d2712
Created: 2026-11-03T05:50:59.108475Z
Model: deepseek-v4.1-flash-1017 / Prompt: policy-v2
Elapsed: 32.414 seconds / Semantic review: completed

Review completed. Findings and proposed changes require human judgment.

Submitted SQL

```
SELECT SUM(total_amount) AS recognized_revenue
FROM orders;
```

Finding 1 / high / metric

Recognized revenue is computed as SUM over all orders regardless of status, including unpaid and cancelled orders, and without excluding orders that have a completed return.

SQL evidence: SUM(total_amount) AS recognized_revenue
FROM orders;

Rule R001: Recognized revenue is SUM(orders.total_amount) for orders with status = 'paid'

Suggestion: Filter to orders.status = 'paid' and exclude orders having a completed return using NOT EXISTS (returns.status = 'completed').

Finding 2 / high / metric

The query sums total_amount but does not exclude orders with a completed return, which inflates recognized revenue.

SQL evidence: FROM orders;

Rule R001: excluding any order with a completed return (returns.status = 'completed')

Suggestion: Add NOT EXISTS subquery excluding orders with returns.status = 'completed'.

Proposed SQL

```
SELECT SUM(o.total_amount) AS recognized_revenue
FROM orders o
WHERE o.status = 'paid'
  AND NOT EXISTS (
      SELECT 1
      FROM returns r
      WHERE r.order_id = o.order_id
        AND r.status = 'completed'
  );
```

Token usage: {"input_tokens": 1076, "output_tokens": 269, "cached_tokens": 0, "estimated_cost": null}

Record 3: passed

ID: b7804c8a-1052-46bc-a02b-ce33b9552f51
Created: 2026-11-03T05:53:25.545206Z
Model: deepseek-v4.1-flash-1017 / Prompt: policy-v2
Elapsed: 3.772 seconds / Semantic review: completed

Review completed. Findings and proposed changes require human judgment.

Parent: be9d1c7c-aed9-4ea0-8118-645b905d2712

Submitted SQL

```
SELECT SUM(o.total_amount) AS recognized_revenue
FROM orders o
WHERE o.status = 'paid'
  AND NOT EXISTS (
      SELECT 1
      FROM returns r
      WHERE r.order_id = o.order_id
        AND r.status = 'completed'
  );
```

No accepted findings were stored. For an incomplete report this does not mean the query passed.

Token usage: {"input_tokens": 1123, "output_tokens": 13, "cached_tokens": 0, "estimated_cost": null}

Record 4: needs_review

ID: 4e655008-c613-4869-b340-01663dd7ac1b
Created: 2026-11-03T05:56:16.478306Z
Model: deepseek-v4.1-flash-1017 / Prompt: policy-v2
Elapsed: 0.001 seconds / Semantic review: skipped

Deterministic checks found issues. Fix these before semantic review.

Submitted SQL

```
SELECT order_id, net_revenue FROM orders;
```

Finding 1 / high / schema

A table or column reference cannot be resolved.

SQL evidence: Column 'net_revenue' could not be resolved. Line: 1, Col: 28

Suggestion: Check column names, aliases and ambiguous references against the schema.

Token usage: not reported; no model request was needed

Shared request context

Commerce Analytics / Demo / version 1 / demo-commerce

```
CREATE TABLE customers (
    customer_id INT PRIMARY KEY, country VARCHAR(2),
    is_test BOOLEAN, signup_at DATETIME
);
CREATE TABLE orders (
    order_id INT PRIMARY KEY, customer_id INT,
    status VARCHAR(20), total_amount DECIMAL(12,2), created_at DATETIME
);
CREATE TABLE payments (
    payment_id INT PRIMARY KEY, order_id INT,
    status VARCHAR(20), amount DECIMAL(12,2), paid_at DATETIME
);
CREATE TABLE returns (
    return_id INT PRIMARY KEY, order_id INT,
    status VARCHAR(20), refunded_amount DECIMAL(12,2)
);
```

R001: Recognized revenue is SUM(orders.total_amount) for orders with status = 'paid', excluding any order with a completed return (returns.status = 'completed'). Pending returns do not exclude an order. Use NOT EXISTS or equivalent logic.

R002: The paid-order count counts distinct orders.order_id with orders.status = 'paid'. Unpaid and cancelled orders must not contribute; returned orders still count as paid orders.

R003: Customer-base metrics (counts of registered customers by country or signup date) exclude test accounts: customers.is_test = 0.

R004: Payment receipts sum payments.amount only where payments.status = 'captured'; pending or failed payments must not contribute. Receipts are separate from recognized revenue.

R005: Refund totals sum returns.refunded_amount only where returns.status = 'completed'. Pending or rejected returns must not contribute.

R006: An order may have multiple payments and multiple returns. Recognized revenue must count each eligible order amount exactly once; joining raw payment or return rows can multiply revenue. SUM(DISTINCT total_amount) is not a valid deduplication because different orders may have equal amounts.

R007: The October 2026 paid-order report uses orders.created_at >= '2026-10-01' AND orders.created_at < '2026-11-01'. DATETIME values on October 31 must be included and November 1 must be excluded.

R008: The paying-customer metric counts distinct orders.customer_id from orders with status = 'paid', not the number of order rows. Multiple paid orders by the same customer count once.